

Lab Window Function

Name: Lê Hồng Anh

MSV: 11247126

Lab 1: Film Ranking

Result

Top 3 longest films per category (using `DENSE_RANK()` with `PARTITION BY` and CTE):

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

```
WITH RankedFilms AS (  
    SELECT  
        c.name AS category_name,  
        f.title,  
        f.length,  
        DENSE_RANK() OVER (PARTITION BY c.name ORDER BY f.length DESC) as length_rank  
    FROM film f  
    JOIN film_category fc ON f.film_id = fc.film_id  
    JOIN category c ON fc.category_id = c.category_id  
)  
SELECT category_name, title, length, length_rank  
FROM RankedFilms  
WHERE length_rank <= 3;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 64

	category_name character varying (25)	title character varying (255)	length smallint	length_rank bigint
1	Action	Worst Banger	185	1
2	Action	Darn Forrester	185	1
3	Action	Casualties Encino	179	2
4	Action	Quest Mussolini	177	3
5	Animation	Pond Seattle	185	1
6	Animation	Gangs Pride	185	1
7	Animation	Theory Mermaid	184	2
8	Animation	Sons Interview	184	2
9	Animation	Lawless Vision	181	3
10	Children	Wrong Behavior	178	1

Lab 2: Growth Analysis (Time Series)

Result

Total revenue per month (using DATE_TRUNC)

Query

Query History

1

SELECT

2

DATE_TRUNC('month', payment_date) AS payment_month,

3

SUM(amount) AS current_revenue

4

FROM payment

5

GROUP BY DATE_TRUNC('month', payment_date)

6

ORDER BY payment_month;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	payment_month timestamp without time zone	current_revenue numeric
1	2007-02-01 00:00:00	8351.84
2	2007-03-01 00:00:00	23886.56
3	2007-04-01 00:00:00	28559.46
4	2007-05-01 00:00:00	514.18

MoM Growth % (using LAG to retrieve previous month's revenue):

Query

Query History

```
1 WITH MonthlyRevenue AS (  
2     SELECT  
3         DATE_TRUNC('month', payment_date) AS payment_month,  
4         SUM(amount) AS current_revenue  
5     FROM payment  
6     GROUP BY DATE_TRUNC('month', payment_date)  
7 )  
8 SELECT  
9     payment_month,  
10    current_revenue,  
11    LAG(current_revenue, 1) OVER (ORDER BY payment_month) AS prev_revenue,  
12    ROUND(  
13        (current_revenue - LAG(current_revenue, 1) OVER (ORDER BY payment_month))  
14        / LAG(current_revenue, 1) OVER (ORDER BY payment_month) * 100  
15    , 2) AS growth_percentage  
16 FROM MonthlyRevenue  
17 ORDER BY payment_month;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows:

	payment_month timestamp without time zone 🔒	current_revenue numeric 🔒	prev_revenue numeric 🔒	growth_percentage numeric 🔒
1	2007-02-01 00:00:00	8351.84	[null]	[null]
2	2007-03-01 00:00:00	23886.56	8351.84	186.00
3	2007-04-01 00:00:00	28559.46	23886.56	19.56
4	2007-05-01 00:00:00	514.18	28559.46	-98.20

Lab 3: Customer Lifetime Value (Running Total)

Result

Cumulative sum of transaction amounts per customer (using SUM() with PARTITION BY and ORDER BY):

Query

Query History

```

1  SELECT
2      customer_id,
3      payment_date,
4      amount,
5      SUM(amount) OVER (
6          PARTITION BY customer_id
7          ORDER BY payment_date
8      ) AS lifetime_value
9  FROM payment
10 ORDER BY customer_id, payment_date;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing

	customer_id smallint 🔒	payment_date timestamp without time zone 🔒	amount numeric (5,2) 🔒	lifetime_value numeric 🔒
1	1	2007-02-14 23:22:38.996577	5.99	5.99
2	1	2007-02-15 16:31:19.996577	0.99	6.98
3	1	2007-02-15 19:37:12.996577	9.99	16.97
4	1	2007-02-16 13:47:23.996577	4.99	21.96
5	1	2007-02-18 07:10:14.996577	4.99	26.95
6	1	2007-02-18 12:02:25.996577	0.99	27.94
7	1	2007-02-21 04:53:11.996577	3.99	31.93
8	1	2007-03-01 07:19:30.996577	4.99	36.92
9	1	2007-03-02 14:05:18.996577	3.99	40.91
10	1	2007-03-02 16:30:04.996577	0.99	41.90
11	1	2007-03-17 11:06:20.996577	4.99	46.89
12	1	2007-03-18 02:25:55.996577	0.99	47.88

✓

Success

Lab 4: The "Top-N" Filter Challenge

Query

```
Query  Query History
1  WITH CustomerSpend AS (
2      SELECT
3          co.country,
4          c.customer_id,
5          c.first_name || ' ' || c.last_name AS customer_name,
6          SUM(p.amount) AS total_spend
7      FROM customer c
8      JOIN address a ON c.address_id = a.address_id
9      JOIN city ci ON a.city_id = ci.city_id
10     JOIN country co ON ci.country_id = co.country_id
11     JOIN payment p ON c.customer_id = p.customer_id
12     GROUP BY co.country, c.customer_id, c.first_name, c.last_name
13 ),
14 RankedCustomers AS (
15
16     SELECT
17         country,
18         customer_name,
19         total_spend,
20         RANK() OVER (PARTITION BY country ORDER BY total_spend DESC) as spend_rank
21     FROM CustomerSpend
22 )
23
24 SELECT
25     country,
26     customer_name,
27     total_spend
28 FROM RankedCustomers
29 WHERE spend_rank = 1
30 ORDER BY country;
```

Result

Top 1 highest-spending customer per country (using CTE with `RANK()` and outer `WHERE rank = 1`):

Data Output Messages Notifications



SQL

	country character varying (50)	customer_name text	total_spend numeric
1	Afghanistan	Vera Mccoy	67.82
2	Algeria	June Carroll	151.68
3	American Samoa	Anthony Schwab	47.85
4	Angola	Claude Herzog	93.80
5	Anguilla	Bobby Boudreau	99.68
6	Argentina	Jordan Archuleta	129.71
7	Armenia	Stephanie Mitch...	118.75
8	Austria	Nora Herrera	113.74
9	Azerbaijan	Raymond Mcwh...	108.75
10	Bahrain	Seth Hannon	108.76
11	Bangladesh	Michelle Clark	146.68
12	Belarus	Clara Shaw	189.60
13	Bolivia	Joel Francisco	92.78
14	Brazil	Marion Snyder	194.61
15	Brunei	Lois Butler	107.66
16	Bulgaria	Tyrone Asher	112.76
17	Cambodia	Elmer Noe	92.76
18	Cameroon	Albert Crouse	96.78
19	Canada	Curtis Irby	167.62
20	Chad	Javier Elrod	122.72
21	Chile	Janet Phillips	124.74